Productivity Tracker

An app to effectively study
and beat procrastination.

Problem Statement

- It’s hard to stay focused in a world full of distractions.
- Social media, games, and random browsing eat up productive time.
- No easy way to track how focused you actually are.
- People want to improve, but don’t know where to start.

SOLUTION — NOCRASTINATOR

A Tool that combines:

- Activity Tracker: Tracks which apps are used and for how long.
- Pomodoro Timer: Customizable sessions with auto-break reminders.
- Focus Score & Streak: Calculated based on how much time you spend in each app category.

Customizable App Categories: Users can edit which apps are productive, unproductive, or neutral.

Tech stack

Tech Stack & Project Modules

Python Packages:

- tkinter, ttk, messagebox – GUI
- threading, time, datetime – Concurrency & Time
- os, sys, csv, json, re – File & Data Handling
- psutil, win32gui, win32process – App Monitoring (Windows)
- plyer.notification – Cross-platform notifications

Custom Modules:

- activity_tracker.py – Tracks active windows & usage
- focus_score.py – Calculates session productivity
- pomodoro.py – Pomodoro timer logic
- config.py – Stores thresholds & constants

Activity Tracker-Technical breakdown

Core Logic:

- State-based Tracking: Detects app/window switches every second.
- Session Logging: Logs duration, title, and productivity status.

Unproductive Time Alert:

- Threshold-Based:
- If T_unproductive $\geqslant$ threshold, alert triggered.
- FSM Logic: Transitions between productive ↔ unproductive states.

Website Detection:

- Regex + Heuristic: Extracts domain via regex + keyword matching.
- Hybrid Classifier: Matches against known productive/unproductive lists.

Output:

- CSV File: Logs timestamp, app, title, duration, productive?
- Usage: Used for productivity graphs, weekly reports, and time insights.

1.Seperates productive and unproductive apps and websites

```python
# First check if app is a browser
if app_name in self.browsers:
    # Extract website from browser window title
    website = self.extract_website_from_title(app_name, window_title)

    if website:
        # Check if website is in productive or unproductive lists
        for prod_site in config.PRODUCTIVE_WEBSITES:
            if prod_site in website:
                print(f"Productive website detected: {website}")
                return True

        for unprod_site in config.UNPRODUCTIVE_WEBSITES:
            if unprod_site in website:
                print(f"Unproductive website detected: {website}")
                return False

        # If website is found but not categorized, log it for future categorization
        print(f"Uncategorized website detected: {website}")
```

2. Calculates the productive and unproductive time(between switching apps too)

```python
ivityTracker:

track_activity_loop(self):
"""Main loop for tracking activity"""
hile self.is_tracking:
   app_name, window_title = self.get_active_window_info()

   if app_name:
       current_time = time.time()

       # If app has changed
       if app_name != self.current_app or window_title != self.current_window_title:
           # Log previous app session if it exists
           if self.current_app and self.app_start_time:
               duration = current_time - self.app_start_time
               is_productive = self.is_productive(self.current_app, self.current_window_title)
               self.log_activity(self.current_app, self.current_window_title, duration, is_productive

           # Start tracking new app
           self.current_app = app_name
           self.current_window_title = window_title
           self.app_start_time = current_time

           # Check if the new app is productive/unproductive/neutral
           is_productive = self.is_productive(app_name, window_title)

           # Track unproductive time across multiple apps
           if is_productive is False:  # Explicitly unproductive
               print(f"Using unproductive app: {app_name}")

               # If this is the first unproductive app in this session
               if not self.is_currently_unproductive:
```

3.Logs the details into a csv file

```python
def log_activity(self, app_name, window_title, duration, is_productive):
    """Log app activity to CSV file"""
    try:
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

        with open(config.ACTIVITY_LOG_FILE, 'a', newline='') as file:
            writer = csv.writer(file)
            writer.writerow([
                timestamp,
                app_name,
                window_title,
                round(duration, 2),
                str(is_productive)
            ])
    except Exception as e:
        print(f"Error logging activity: {e}")
```

FocusScore Module - Productivity Metrics

Purpose:

Tracks daily productivity using app usage data to calculate a focus score (0–100), analyze trends, and provide improvement suggestions.

Key Features:

- 📊 Daily Focus Score: Based on productive vs. unproductive time
- 🔁 Streak Tracking: Counts consecutive productive days
- 📅 Weekly Analysis: Avg. score, most productive day, time spent
- 💡 Smart Suggestions: Trends, distractions, and focus tips

Score Logic:

Weighted formula using productive_time & unproductive_time with customizable weights.

Suggestions Example:

- "Productivity improving – keep going!"
- "Spent 2+ hrs on Instagram – consider reducing time."
- "Try Pomodoro technique for better focus."

1. Calculates daily scores

```python
def calculate_daily_score(self, date=None):
    """
    Calculate focus score for a given date.
    Score is based on productive vs. unproductive time.
    """
    if date is None:
        date = datetime.now().strftime('%Y-%m-%d')
    if date in self.scores:
        return self.scores[date]['score']
    total_time = 0
    productive_time = 0
    unproductive_time = 0
    neutral_time = 0
    try:
        with open(config.ACTIVITY_LOG_FILE, 'r', newline='') as file:
            reader = csv.reader(file)
            next(reader)
            for row in reader:
                if row[0].startswith(date):
                    duration = float(row[3])
                    is_productive = row[4]
                    total_time += duration
                    if is_productive == 'True':
                        productive_time += duration
                    elif is_productive == 'False':
                        unproductive_time += duration
                    else:
                        neutral_time += duration
    except Exception as e:
        print(f"Error reading activity data: {e}")
```

2. Generates suggestions based on time spent

```python
def _generate_suggestions(self):
    # Suggestion 1: Productivity trend
    if recent_scores and len(recent_scores) >= 3:
        avg_first_half = sum(recent_scores[:len(recent_scores)//2]) / (len(recent_scores)//2)
        avg_second_half = sum(recent_scores[len(recent_scores)//2:]) / (len(recent_scores) - len(recent_scores)//2)

        if avg_second_half > avg_first_half:
            suggestions.append("Your productivity is trending upward. Keep up the good work!")
        elif avg_second_half < avg_first_half:
            suggestions.append("Your productivity has been declining. Try to focus on more productive tasks.")

    # Suggestion 2: Most distracting apps
    if unproductive_apps:
        sorted_apps = sorted(unproductive_apps.items(), key=lambda x: x[1], reverse=True)
        top_distraction = sorted_apps[0][0]
        top_distraction_time = sorted_apps[0][1] / 3600  # Convert to hours

        if top_distraction_time > 1:
            suggestions.append(f"You spent {top_distraction_time:.1f} hours on {top_distraction}. Consider limiting ti

    # Suggestion 3: General suggestions
    if not recent_scores or sum(recent_scores) / len(recent_scores) < 50:
        suggestions.append("Try using the Pomodoro technique to improve focus and productivity.")

    return suggestions
```

3. Calculates streak:

```python
def get_streak(self):
    """Calculate the current productivity streak"""
    min_score = 50  # Minimum score to count as productive day

    # Sort dates in reverse order (newest first)
    dates = sorted(self.scores.keys(), reverse=True)

    if not dates:
        return 0

    streak = 0
    today = datetime.now().date()

    for i, date_str in enumerate(dates):
        date = datetime.strptime(date_str, '%Y-%m-%d').date()
        score = self.scores[date_str]['score']

        # Check if this date is part of continuous streak
        expected_date = today - timedelta(days=i)

        if date == expected_date and score >= min_score:
            streak += 1
        else:
            break

    return streak
```

PomodoroTimer Module - Focus & Break Manager

Purpose:

Implements a customizable Pomodoro timer with work and break phases to improve focus and prevent burnout.

Key Features:

- ▶ Start, Pause, Resume, Stop functions
- 🔁 Cycle Management: Auto-switches between work, short break, and long break.

Cycle Logic:

After N work sessions → Long Break

Else → Short Break

Then → Work again

1.1. Initialization & Configuration

```python
class PomodoroTimer:
    """
    Implements a Pomodoro timer with work sessions and breaks.
    Supports pausing, resuming, and notifications.
    """

    def __init__(self):
        self.work_duration = config.POMODORO_WORK_DURATION * 60  # Convert to seconds
        self.short_break_duration = config.POMODORO_SHORT_BREAK_DURATION * 60
        self.long_break_duration = config.POMODORO_LONG_BREAK_DURATION * 60
        self.cycles_before_long_break = config.POMODORO_CYCLES_BEFORE_LONG_BREAK

        self.current_cycle = 0
        self.time_remaining = self.work_duration
        self.current_phase = "Ready"
        self.is_running = False
        self.is_paused = False
        self.timer_thread = None

        # Callback functions
        self.on_tick = None  # Called every second with time update
        self.on_phase_change = None  # Called when phase changes (work → break)
        self.on_complete = None  # Called when a full pomodoro cycle completes

        print("Pomodoro timer initialized")
```

2. Timer Control & Main Loop

```python
def pause(self):
    """Pause the timer"""
    if self.is_running and not self.is_paused:
        self.is_paused = True
        print("Pomodoro timer paused")

def resume(self):
    """Resume the timer"""
    if self.is_running and self.is_paused:
        self.is_paused = False
        print("Pomodoro timer resumed")

def stop(self):
    """Stop the timer"""
    self.is_running = False
    if self.timer_thread:
        self.timer_thread.join(timeout=1)

    self.current_cycle = 0
    self.time_remaining = self.work_duration
```

3. Phase Handling & Notifications

```python
def _handle_phase_complete(self):
    """Handle completion of a Pomodoro phase"""
    # Determine the next phase
    if self.current_phase == "Work":
        self.current_cycle += 1

        # Decide if we need a long break or short break
        if self.current_cycle % self.cycles_before_long_break == 0:
            self.current_phase = "Long Break"
            self.time_remaining = self.long_break_duration
            self._notify("Time for a long break!", "Take a longer break to recharge.")
        else:
            self.current_phase = "Short Break"
            self.time_remaining = self.short_break_duration
            self._notify("Time for a short break!", "Take a quick break to refresh.")

    elif self.current_phase in ["Short Break", "Long Break"]:
        self.current_phase = "Work"
        self.time_remaining = self.work_duration
        self._notify("Break over!", "Time to get back to work.")

    # Call phase change callback
    if self.on_phase_change:
        self.on_phase_change(self.current_phase)

    # Call complete callback if we finished a full cycle
    if self.current_phase == "Work" and self.on_complete:
        self.on_complete(self.current_cycle)
```

```python
def _notify(self, title, message):
    """Send a notification about Pomodoro phase change"""
    notification.notify(
        title=title,
        message=message,
        timeout=10
    )
    print(f"Notification: {title} - {message}")

def get_time_remaining_str(self):
    """Get the remaining time as a formatted string (MM:SS)"""
    minutes = self.time_remaining // 60
    seconds = self.time_remaining % 60
    return f"{minutes:02d}:{seconds:02d}"

def get_progress(self):
    """Get the progress as a percentage (0-100)"""
    if self.current_phase == "Work":
        total_time = self.work_duration
    elif self.current_phase == "Short Break":
        total_time = self.short_break_duration
    elif self.current_phase == "Long Break":
        total_time = self.long_break_duration
    else:
        return 0

    time_elapsed = total_time - self.time_remaining
    return (time_elapsed / total_time) * 100 if total_time > 0 else 0
```

PomodoroTimer Module - Focus & Break Manager

Core Functionalities

- Pomodoro Timer: Start, pause/resume, and reset cycles for focused work.
- Live Activity Tracking: Monitors active windows/apps with productivity classification (Productive / Unproductive / Neutral).
- Alerts: Sends warnings after extended unproductive app usage.

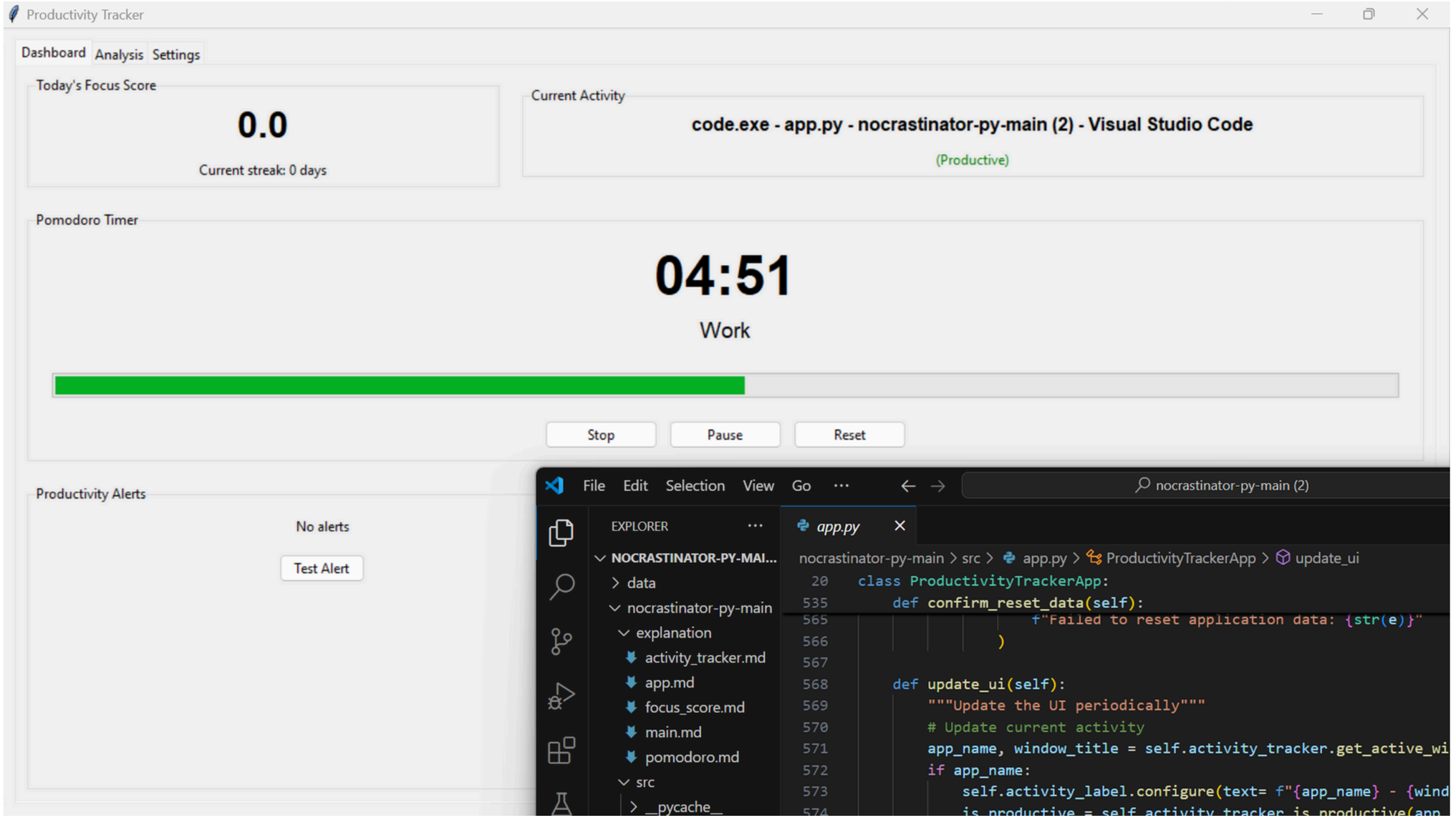
Insights & Feedback

- Weekly Focus Analysis: Displays date-wise focus scores, productive & unproductive hours.
- Productivity Trend: Highlights improvement, stability, or decline in focus.
- Top App Usage: Shows most used apps with duration & productivity color codes.
- Statistics & Suggestions: Daily stats + personalized improvement tips.

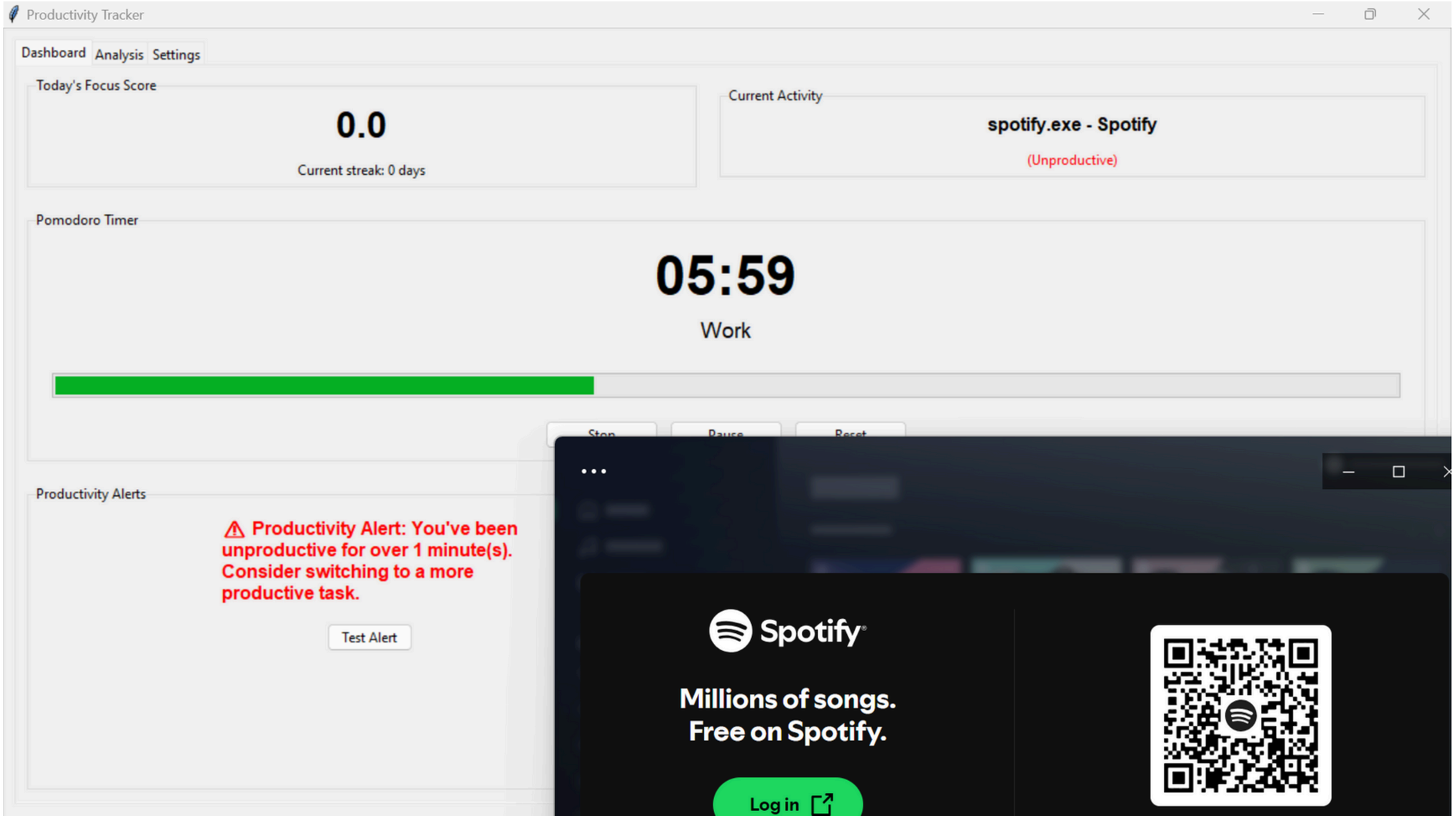
Data Control & System Utilities

- Data Reset Option: Erase all stored logs and restart the app.
- Phase Tips: Guidance for each Pomodoro phase (Work / Short Break / Long Break).
- Auto UI Update: Keeps interface and labels refreshed in real time.

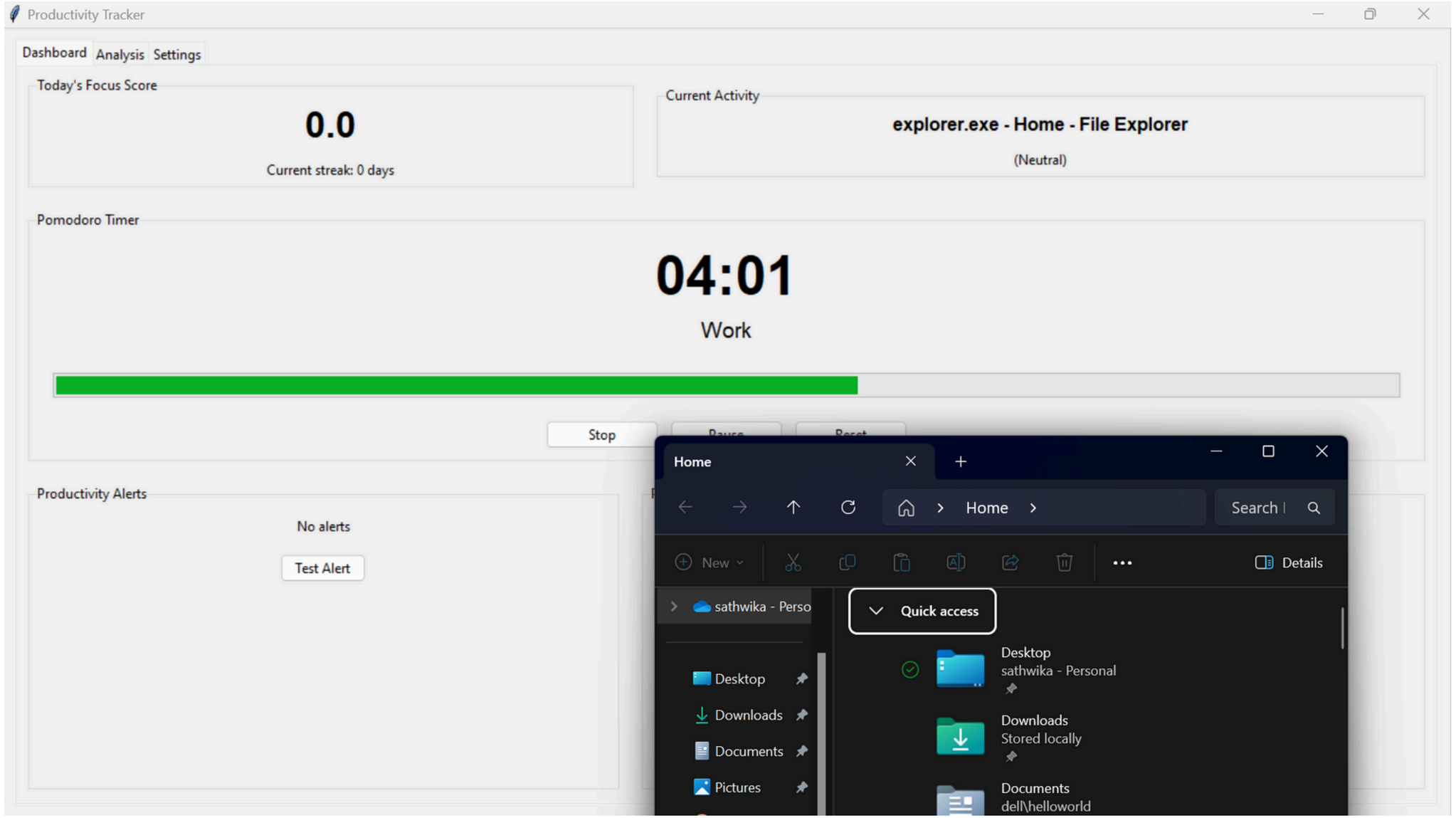
What it looks like when an productive app is running.



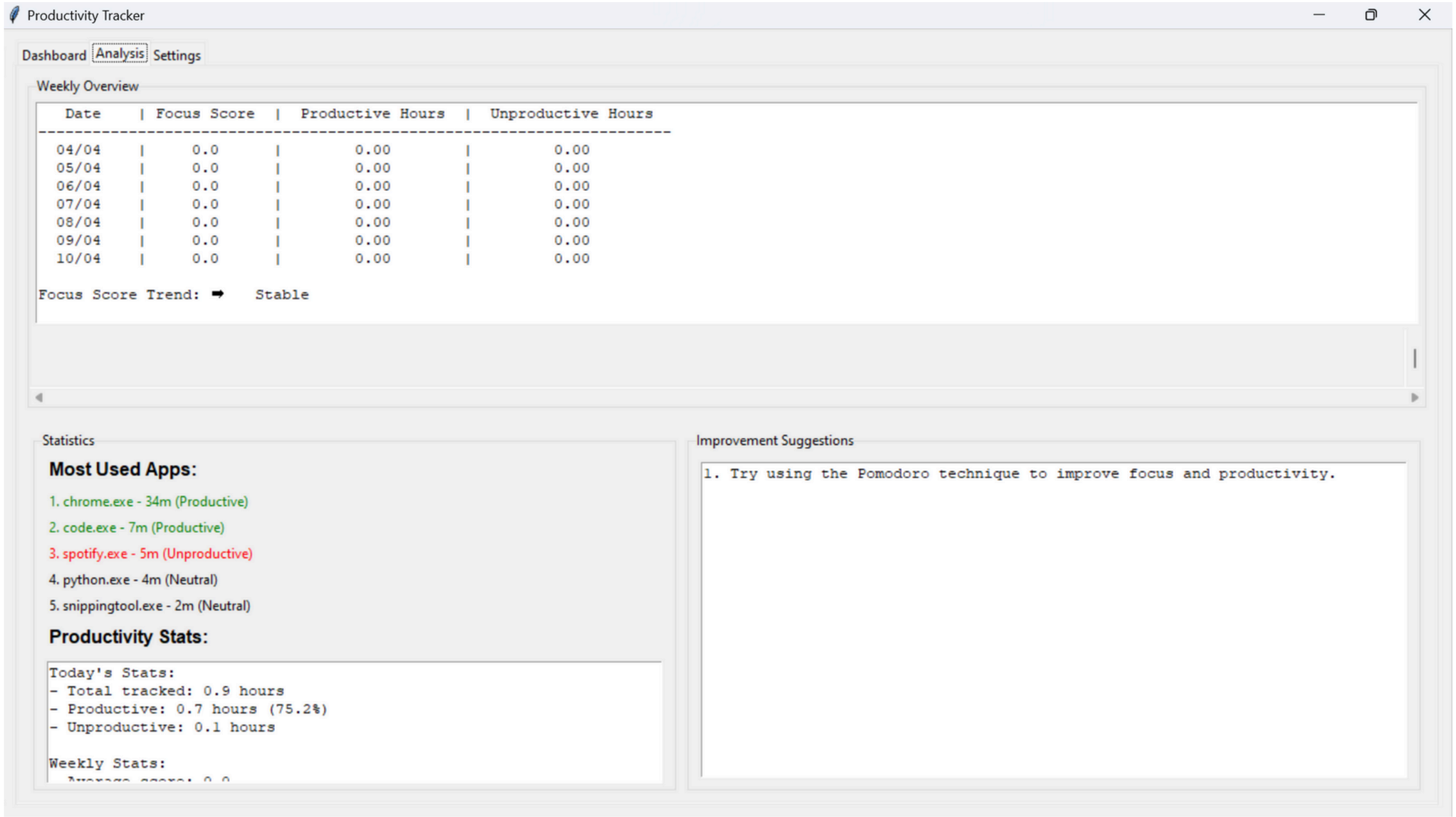
What it looks like when an unproductive app is running.



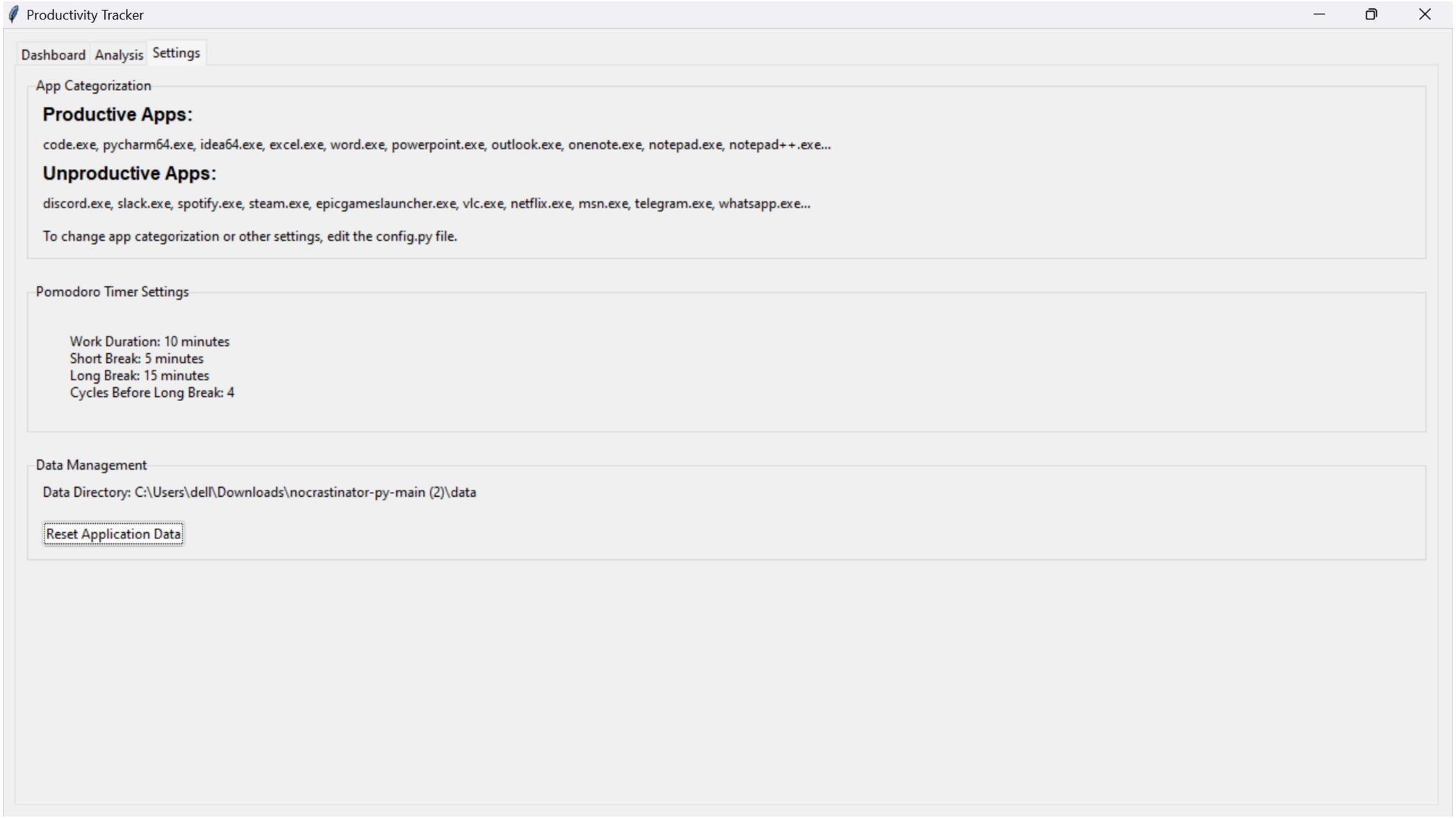
Dashboard


Productivity Tracker
Dashboard
Analysis
Settings
Today's Focus Score
0.0
Current streak: 0 days
Current Activity
explorer.exe - Home - File Explorer
(Neutral)
Pomodoro Timer
04:01
Work
Stop
Productivity Alerts
No alerts
Test Alert
Home
Search
New
Details
sathwika - Perso
Quick access
Desktop
Downloads
Documents
Pictures
Desktop
sathwika - Personal
Downloads
Stored locally
Documents
dell\helloworld

Analysis


Productivity Tracker
Dashboard
Analysis
Settings
Weekly Overview
Date | Focus Score | Productive Hours | Unproductive Hours
04/04 | 0.0 | 0.00 | 0.00
05/04 | 0.0 | 0.00 | 0.00
06/04 | 0.0 | 0.00 | 0.00
07/04 | 0.0 | 0.00 | 0.00
08/04 | 0.0 | 0.00 | 0.00
09/04 | 0.0 | 0.00 | 0.00
10/04 | 0.0 | 0.00 | 0.00
Focus Score Trend: ➡ Stable
Statistics
Most Used Apps:
1. chrome.exe - 34m (Productive)
2. code.exe - 7m (Productive)
3. spotify.exe - 5m (Unproductive)
4. python.exe - 4m (Neutral)
5. snippingtool.exe - 2m (Neutral)
Productivity Stats:
Today's Stats:
- Total tracked: 0.9 hours
- Productive: 0.7 hours (75.2%)
- Unproductive: 0.1 hours
Weekly Stats:
Improvement Suggestions
1. Try using the Pomodoro technique to improve focus and productivity.

Settings


Productivity Tracker
Dashboard
Analysis
Settings
App Categorization
Productive Apps:
code.exe, pycharm64.exe, idea64.exe, excel.exe, word.exe, powerpoint.exe, outlook.exe, onenote.exe, notepad.exe, notepad++.exe...
Unproductive Apps:
discord.exe, slack.exe, spotify.exe, steam.exe, epicgameslauncher.exe, vlc.exe, netflix.exe, msn.exe, telegram.exe, whatsapp.exe...
To change app categorization or other settings, edit the config.py file.
Pomodoro Timer Settings
Work Duration: 10 minutes
Short Break: 5 minutes
Long Break: 15 minutes
Cycles Before Long Break: 4
Data Management
Data Directory: C:\Users\dell\Downloads\nocrastinator-py-main (2)\data
Reset Application Data

THANK YOU!